Day 128 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 19, Day 3 reference

Watch: 88 - Applications of Autoencoders - Anomaly Detection (DigitalSreeni) | Anomaly Detection Using Autoencoders | Tensorflow | Keras (MLWorks)

`day128_anomaly_detection.py`, segment by segment, in the order `main()` uses things. The anomaly scores, the threshold and its statistics are new; Day 126's network, Day 123's PCA, Day 119's distances and Day 117's metrics are imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: s = anomaly scores · y or a = 1 for anomalies, 0 for normal · t = a threshold · tr, te, val = training, test and threshold-setting rows

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
for w in ("week-18-unsupervised-learning-i", "week-17-classical-ml-kernels-ensembles"):
    sys.path.insert(0, os.path.join(HERE, "..", w))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day126_exercise as d126
    import day126_autoencoders as d126m
    import day123_exercise as d123
    import day119_exercise as d119
    import day117_exercise as d117
ACTS = ["relu", "linear", "relu", "linear"]
SIZES = [784, 256, 32, 256, 784]
```

- `for w in (...): sys.path.insert(0, os.path.join(HERE, "..", w))` — a loop adds two earlier weeks' folders to the search path.
- `import day117_exercise as d117` — Day 117's ROC AUC and average precision, unchanged.

Segment 1 — Scores and thresholds

The three scores

MATH

$s_{AE}(x) = |x - decoder(encoder(x))|^2; \quad s_{NN}(x) = min_r |x - r|$

CODE

```
def ae_score(net, X, acts, kind="bce"):
    return ((d126.reconstruct(net, X, acts, kind) - X) ** 2).sum(1)
def pca_score(X, pca, k):
    mu, V = pca
    return ((d123.reconstruct(d123.transform(X, mu, V, k), mu, V, k) - X) ** 2).sum(1)
def nn_score(X, ref, chunk=2000):
    return np.concatenate([np.sqrt(np.maximum(d119.sqdist(X[s:s + chunk], ref), 0).min(1)) for s
        in range(0, len(X), chunk)])
```

- `((reconstruct(...) - X) ** 2).sum(1)` — squared error summed over each example's pixels: one score per row.
- `np.maximum(d119.sqdist(...), 0).min(1)` — each row's nearest reference; maximum guards against tiny negative rounding.

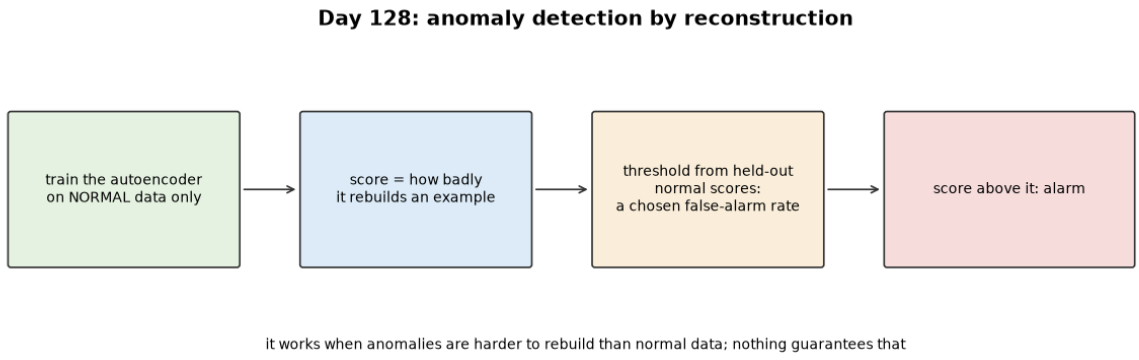
threshold and alarm_stats

CODE

```
def threshold(normal_scores, false_alarm):
    return float(np.quantile(normal_scores, 1 - false_alarm))
def alarm_stats(scores, y, t):
    flag = scores > t; tp = int((flag & (y == 1)).sum()); fp = int((flag & (y == 0)).sum())
    return {"recall": tp / max((y == 1).sum(), 1), "false alarms": fp / max((y == 0).sum(), 1),
        "precision": tp / max(flag.sum(), 1)}
```

- `np.quantile(normal_scores, 1 - false_alarm)` — the score below which 1 – false_alarm of normal examples fall.
- `flag & (y == 1)` — & is element-wise AND on True/False arrays: flagged and truly anomalous.
- `max(..., 1)` — avoids dividing by zero when nothing is flagged.

SYSTEM DIAGRAM



Train on normal data, score, threshold.

Segment 2 — The experiments

demo_one: training without the held-out digit

CODE

```
tr = np.where(y[:60000] != held)[0][:20000]; te = np.arange(60000, 70000); a = (y[te] ==
    held).astype(int)
t0 = time.perf_counter(); net = train(X[tr]); t1 = time.perf_counter() - t0
```

- `np.where(y[:60000] != held)[0][:20000]` — the positions of training images of other digits, the first 20,000 of them.
- `(y[te] == held).astype(int)` — 1 for the held-out digit (the anomalies), 0 for the rest.

demo_contaminated: hiding anomalies in the training set

CODE

```
k = int(round(10000 * share)); tr = np.r_[normal[:10000 - k], nines[:k]]
```

- `np.r_[a, b]` — joins two index arrays: the training set stays at 10,000 images, k of them 9s.

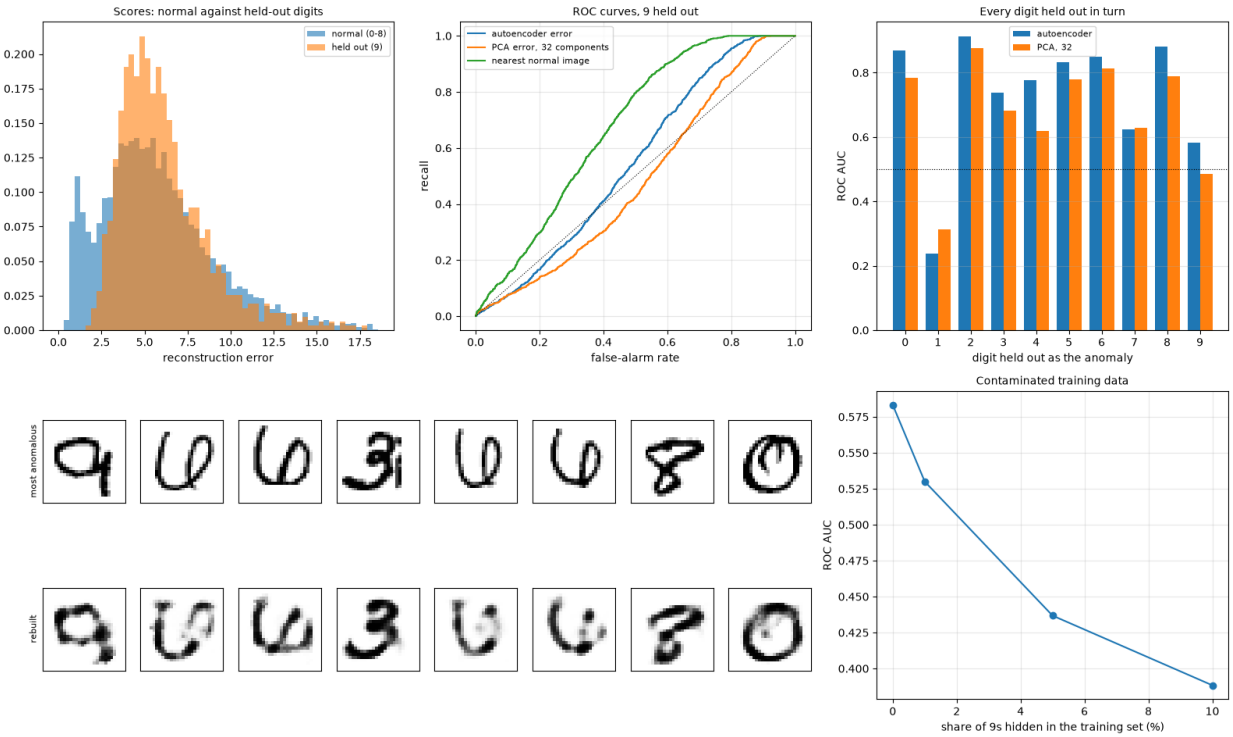
demo_tumours: scaling with benign rows only

CODE

```
mu, sd = X[tr].mean(0), X[tr].std(0) + 1e-12; Z = ((X - mu) / sd).astype(np.float32)
```

- `X[tr].mean(0)` — the scaling is learned from the 200 benign training tumours alone, so no test or malignant row leaks in.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.quantile(a, q)` — the value below which a share q falls.
- `a & b` — element-wise AND.
- `np.where(cond)[0]` — positions where cond is True.
- `np.r_[a, b]` — join arrays end to end.
- `np.argsort(-s)[:8]` — the eight largest scores.
- `for w in (...): sys.path.insert(...)` — several folders added in a loop.

End of Day 128 syntax reference. For the experiments and what they showed, see the Day 128 complete guide.